

CASCADEIS

COMPILADORES 1 • P1 • Grupo 05

OBJETIVO

Traduzir um subconjunto de C para Python

ENTREGA ATUAL

Análise léxica e sintática



TIPO DE PROJETO

Compilador

Linguagem escolhida: C

INTEGRANTES

- Ana Caroline Dantas da Silva
- Felipe Matheus Ribeiro Lopes
- Gabriel Goldenberg Moita
- Luiz Gustavo Silva de Almeida
- Pedro Araujo Lucena

Escopo: construções de C suportadas e limites

Escopo do compilador

CONSTRUÇÕES SUPORTADAS

TIPOS PRIMITIVOS

```
int, float e char;  
void para retorno de funções
```

CONTROLE DE FLUXO

```
if, else, while, for
```

FUNÇÕES

```
declaração, definição, chamada,  
return
```

OPERADORES

```
aritméticos: +, -, *, /;  
atribuição: =;  
relacionais: ==, !=, <, >, <=, >=;  
lógicos: &&, ||, !
```

LITERAIS E IDENTIFICADORES

```
Inteiros, decimais, caracteres e  
strings;  
[a-zA-Z_][a-zA-Z0-9_]*
```

DELIMITADORES E COMENTÁRIOS

```
{}, (), vírgula, ponto e vírgula;  
// e /* ... */
```

LIMITES DO ESCOPO

O compilador trata um subconjunto de C focado em lógica imperativa básica, operações matemáticas e controle de fluxo.

FOCO DE IMPLEMENTAÇÃO

Análise e execução de construções imperativas essenciais.

NÃO É C COMPLETO!

O escopo é deliberadamente limitado ao subconjunto descrito.

Metodologia e sprints realizadas

METODOLOGIA ADOTADA

Scrum ciclos de trabalho curtos (Sprints)

SPRINTS REALIZADAS

2 sprints concluídas

Sprint 1

- ambiente configurado
- repositório estruturado
- subconjunto de C e especificação definidos
- protótipo Flex/Bison

Sprint 2

- lexer finalizado
- parser inicial consolidado
- integração Flex-Bison-GCC
- testes válidos e inválidos

| Colaboração por issues, branches, pull requests e milestones

Organização das entregas no GIT



planejam
e organizam
os objetivos
de cada entrega

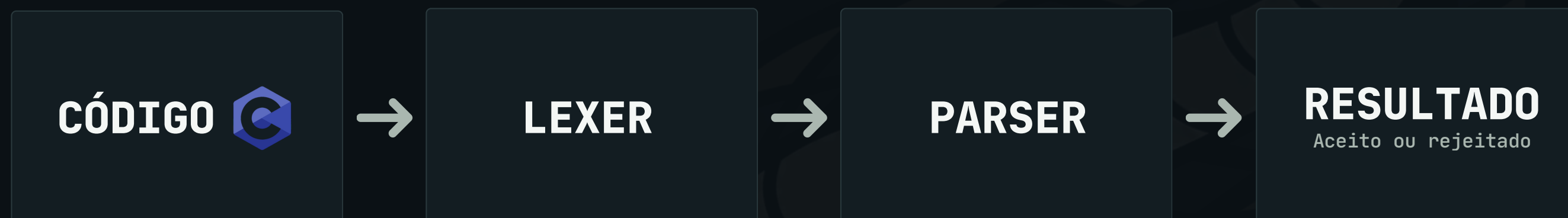
registram
tarefas,
responsáveis
e critérios
de aceite

separam o
desenvolvimento
das tarefas

reúnem
alterações
para revisão
e integração
na main

O QUE ESTÁ FUNCIONANDO

Pipeline atual do compilador



LEXER COMPLETO

Reconhece os tokens do subconjunto de C e informa erros léxicos com indicação de linha

PARSER INICIAL

Verifica a estrutura sintática, com precedência definida e sem conflitos não resolvidos.

INTEGRAÇÃO

O main.c coordena a análise e informa se o código foi aceito ou rejeitado.

AUTOMATIZAÇÃO

Makefile automatiza a compilação e a execução dos testes válidos e inválidos.

Analisador Léxico com FLEX

O QUE FAZ

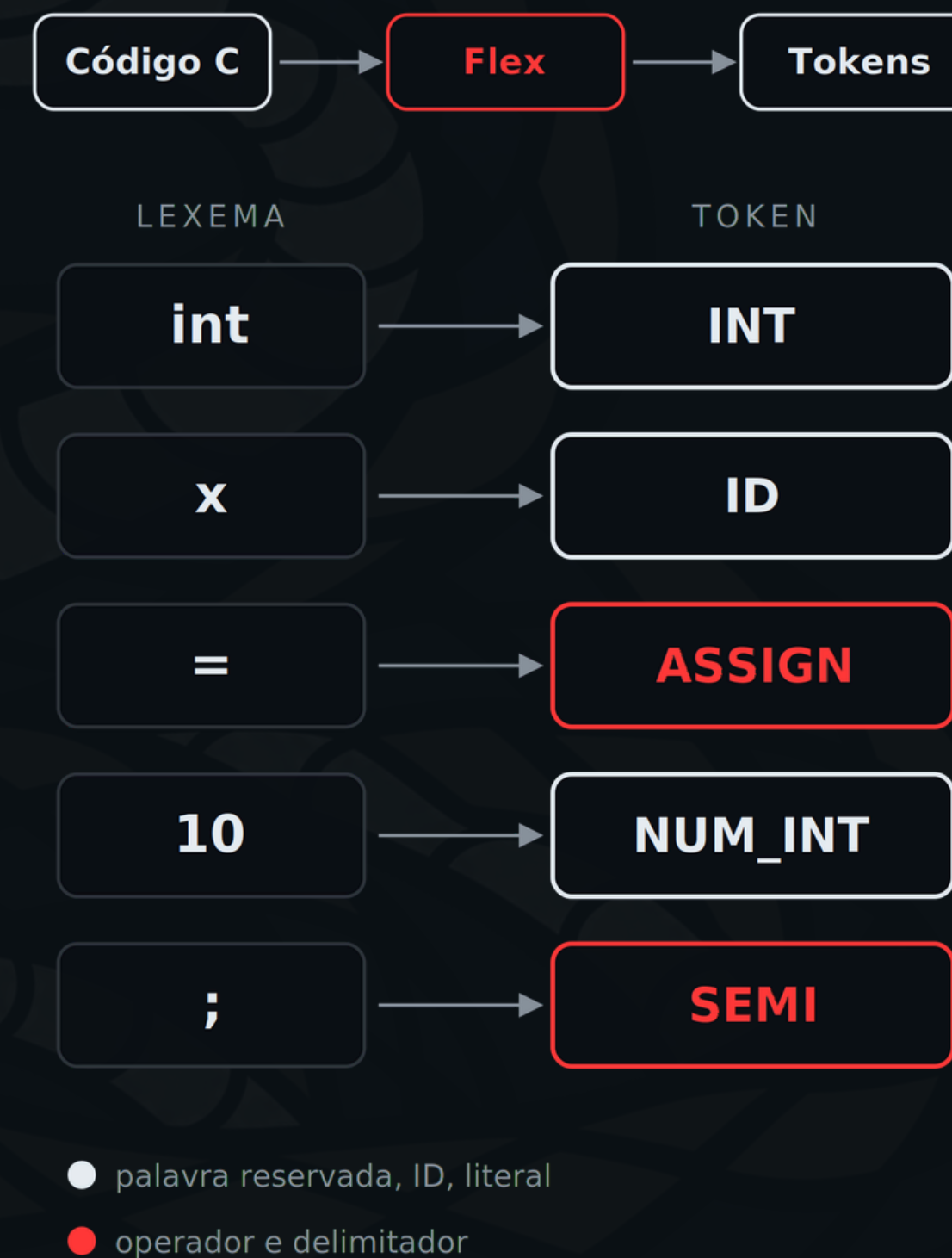
- Transforma o código C em tokens: `int x = 10;` → `INT ID ASSIGN NUM_INT SEMI`
- Ignora espaços e comentários e entrega os tokens ao parser (Bison)

TOKENS RECONHECIDOS

- Palavras reservadas, identificadores, literais (`int`, `float`, `char`, `string`)
- Operadores (`+` `-` `*` `/`, `==` `!=` `<` `>` `<=` `>=`, `&&` `||` `!`, `=`) e delimitadores (`{` `}` `(` `)` `,` `;`)

RECURSOS DO FLEX

- `yylineno`: linha atual para as mensagens de erro
- Estados exclusivos `COMENTARIO` e `STRING_STATE`: tratam comentários de bloco e strings
- Regra do maior casamento: `==` não vira dois `=`, e `int` é palavra reservada, não `ID`



Analisador Sintático com BISON

O QUE FAZ

- Recebe os tokens do Flex e verifica se formam um programa válido pela gramática
- Rejeita estruturas inválidas e informa a linha do erro

GRAMÁTICA

- Programa: sequência de funções (declaração ou definição)
- Comandos: declaração, atribuição, chamada, **if/else**, **while**, **for**, **return**
- Expressões: literais, IDs, chamadas, parênteses e operadores

RECURSOS DO BISON

- **%token**: 34 tokens, os mesmos que o Flex retorna
- **%left** / **%right**: precedência e associatividade, de **||** (menor) até **!** e **-** unário (maior)
- **%expect 0**: o build falha se houver qualquer conflito na gramática
- **parse.error verbose**: mensagens como **unexpected IF**

DERIVAÇÃO DE $2 + 3 * 4$



*** tem precedência maior que +**
então $3 * 4$ vira sub-expressão primeiro

Testes do repositório

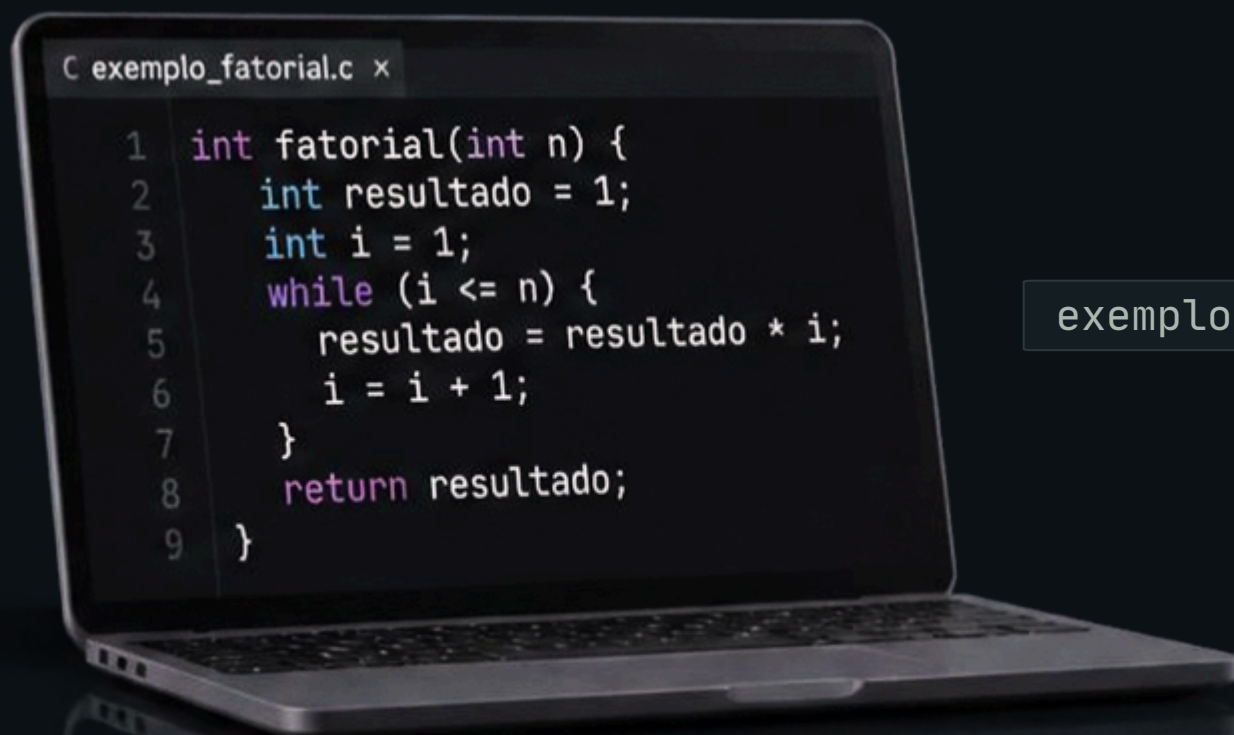
Testes válidos e inválidos

Válidos

- exemplo_fatorial.c
- exemplo_laco_for.c
- identificadores_literais.c
- operadores_delimitadores.c
- palavras_reservadas_tipos.c

Inválidos

- caracteres_nao_reconhecidos.c
- comentario_nao_terminado.c
- diretivas_nao_suportadas.c
- erro_sintatico_bloco.c
- identificador_invalido.c
- operador_atribuicao_composta.c
- operador_bitwise_and.c
- operador_decremento.c
- operador_incremento.c
- operador_shift_esquerda.c
- string_nao_terminada.c



exemplo_fatorial.c

Makefile

Automatiza a geração, compilação e a execução dos testes.

ESTRUTURA DO MAKEFILE

- Variáveis definem ferramentas, caminhos e arquivos de saída.
- Um bloco condicional adapta os comandos ao Windows e ao Linux.
- Regras associam um alvo às suas dependências e aos comandos a executar.
- **all: build test** define a tarefa padrão.

O QUE ACONTECE EM BUILD

- 1 Bison lê `parser.y` e gera `parser.tab.c` e `parser.tab.h`.
- 2 Flex lê `lexer.l` e gera `flex.yy.c`.

GCC compila os arquivos gerados com `main.c`.
- 3 Cria `lexer_test.exe` no **Windows** ou `lexer_test` no **Linux**.

COMANDOS NO TERMINAL

- **make** ou **make all**
Compila e executa todos os testes.
- **make build**
Gera os arquivos e compila o analisador.
- **make run**
Compila e abre a entrada manual.
- **make test**
Compila e executa os dois grupos de testes.
- **make test-validos**
Executa os exemplos que devem ser aceitos.
- **make test-invalidos**
Executa os exemplos que devem ser rejeitados.
- **make clean**
Apaga o executável e os arquivos gerados.

OBSERVAÇÃO

A versão atual analisa o código C e informa **aceitação** ou **rejeição**. Não executa o programa C nem gera Python ainda.

DIFICULDADES E MELHORIAS

Desafios enfrentados e soluções implementadas

DIFICULDADES

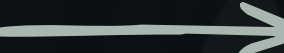
1

Reconhecimento de
entradas inválidas



2

Isolamento e diagnóstico
das falhas



3

Integração entre
Flex e Bison



MELHORIAS IMPLEMENTADAS

1

Regras específicas no lexer para
identificadores iniciados por números e
para o operador não suportado `--`

2

Separação dos testes inválidos por caso e
conferência das mensagens de erro e dos
códigos de saída.

3

Uso do cabeçalho compartilhado
'`parser.tab.h`' para manter a interface
entre lexer, parser e programa principal.

Melhorias adicionais: controle de conflitos com `%expect 0` e fechamento do
arquivo de entrada após a análise.

PRÓXIMAS ENTREGAS

Rumo à tradução C → Python

REGRAS SINTÁTICAS

Finalizar as regras no Bison e consolidar o reconhecimento das construções do subconjunto de C.

AST E TABELA DE SÍMBOLOS

Implementar a árvore sintática abstrata e a estrutura para registrar informações sobre os identificadores.

ANÁLISE SEMÂNTICA

Iniciar as verificações de significado do programa, como uso de identificadores e compatibilidade de tipos.

AMPLIAÇÃO DOS TESTES

Adicionar casos sintáticos e semânticos para validar as próximas implementações.



FIM

